Informe Técnico: Sistema RAG Local con LM Studio y LangChain

Asignatura: Sistemas de Aprendizaje Automático + Modelos de Inteligencia Artificial

Fecha: 24 de mayo de 2024

Autor: [Tu Nombre Completo]

Centro: IES Ribera de Castilla

1. Descripción del Sistema

El presente proyecto implementa un sistema de **Generación Aumentada por Recuperación (RAG)** ejecutado al 100% en entorno local. El sistema permite a los usuarios interactuar mediante lenguaje natural con una base de conocimientos privada, eliminando la dependencia de servicios en la nube y garantizando la privacidad de los datos.

Para esta práctica, el sistema ha sido alimentado con un conjunto de manuales técnicos y guías de proyectos (incluyendo el documento `Practica_Final_Webots.pdf`), permitiendo resolver dudas específicas sobre procedimientos internos y arquitecturas de simulación.

2. Arquitectura Implementada

El sistema se organiza en cinco capas funcionales:

1. **Ingesta (Document Loader):** Se utiliza `PyPDFLoader` y `TextLoader` para leer archivos en crudo. El texto se procesa mediante un `RecursiveCharacterTextSplitter` .
2. **Embeddings:** Se emplea el modelo local `all-MiniLM-L6-v2` de HuggingFace, que transforma el texto en vectores de 384 dimensiones.
3. **Vectorstore:** Se utiliza `ChromaDB` como base de datos vectorial persistente, almacenando los documentos en el directorio local `./chroma_db` .
4. **Cadena RAG:** Una cadena `RetrievalQA` orquesta la comunicación entre el buscador y el modelo de lenguaje, inyectando los fragmentos recuperados en un prompt personalizado.
5. **Agente e Interfaz:** Un agente de tipo `ReAct` gestiona tres herramientas (*tools*): consulta documental, consulta de hora del sistema y un contador de frecuencia de palabras clave.

3. Justificación del Modelo y Parámetros

3.1 Modelo LLM: Phi-3-mini-4k-instruct

Se ha seleccionado **Phi-3-mini** debido a su excepcional equilibrio entre rendimiento y requisitos de hardware. Al trabajar en un entorno con recursos optimizados, este modelo de 3.8B parámetros ofrece una latencia baja y una capacidad de razonamiento lógica suficiente para seguir el formato ReAct de los agentes.

3.2 Parámetros de Indexación

- **chunk_size (500):** Se eligió este valor para asegurar que cada fragmento contenga una idea completa pero sea lo suficientemente pequeño como para que el LLM reciba varios contextos distintos sin exceder su ventana de contexto.
- **chunk_overlap (50):** Este solapamiento del 10% garantiza que no se pierda información importante que pueda quedar cortada justo en el límite de un chunk.

3.3 Parámetros de Recuperación y Generación

- **k (4):** Recuperar los 4 fragmentos más relevantes permite al modelo contrastar información de distintas partes del documento, aumentando la precisión de la respuesta.
- **Temperature (0.1):** Se ha configurado una temperatura muy baja para minimizar la creatividad del modelo y priorizar la fidelidad al texto original, reduciendo el riesgo de alucinaciones.

4. Resultados y Evaluación

Pregunta	Respuesta del Sistema	Evaluación
"¿Cuál es el tema principal?"	"Descripción del problema, solución adoptada, capturas de simulación..."	**Excelente:** Identificó correctamente los apartados del PDF.
"¿Qué hora es?"	"Friday, 24 de May de 2024, 18:30"	**Correcto:** El agente utilizó la Tool de tiempo con éxito.
"¿Cuántas veces aparece 'webots'?"	"La palabra 'webots' aparece aproximadamente 12 veces..."	**Correcto:** Se ejecutó la Tool personalizada de búsqueda.
"¿Cómo configurar GPT-4?"	"No tengo información sobre eso."	**Correcto:** El sistema se limitó al contexto (Guardrail).

5. Limitaciones y Mejoras

Limitaciones

- **Velocidad de Inferencia:** Al ejecutarse en CPU, el tiempo de pensamiento del agente (razonamiento ReAct) puede tardar entre 5 y 10 segundos.
- **Contexto General:** El sistema tiene dificultades para responder preguntas que requieran una síntesis de todo el documento si este es muy extenso.

Mejoras Futuras

1. **Memoria Conversacional:** Implementar `ConversationBufferMemory` para que el agente recuerde preguntas anteriores del usuario.
2. **Interfaz Gráfica:** Desarrollar una UI con Streamlit o Gradio para mejorar la experiencia de usuario.
3. **Reranking:** Añadir una capa de re-clasificación de resultados para mejorar la relevancia de

los chunks recuperados antes de enviarlos al LLM.

6. Conclusiones

La implementación de un sistema RAG local demuestra que es viable desplegar soluciones de IA generativa potentes sin depender de infraestructuras externas. El uso de LangChain ha facilitado la integración de herramientas diversas, permitiendo que el modelo no solo "lea", sino que "actúe" sobre el entorno local.